

**Universitatea  
Transilvania  
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ  
ȘI ȘTIINȚA CALCULATOARELOR**

# **Grafică 3D implementată pe FPGA**

Conducător științific:

Șl. Dr. Ing. ZAHARIA Corneliu

Absolvent:

BRAȘOVEANU Petru-Andrei

**Brașov, 2026**

# Cuprins

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Lista de acronime                                               | 3  |
| 1 Introducere                                                   | 4  |
| 1.1 Tema proiectului                                            | 4  |
| 1.2 Obiectivele principale                                      | 4  |
| 1.2.1 Obiective hardware                                        | 4  |
| 1.2.2 Obiective de documentație                                 | 5  |
| 1.3 Motivație                                                   | 5  |
| 2 Studiu asupra temei alese                                     | 6  |
| 2.1 Reprezentarea în virgulă fixă                               | 6  |
| 2.2 Stadiul actual din punct de vedere comercial                | 7  |
| 2.2.1 Pipeline-ul grafic în GPU-urile moderne                   | 7  |
| 2.2.2 FPGA-urile ca platformă pentru grafică 3D                 | 8  |
| 2.2.3 MiSTer FPGA – reimplementarea hardware-ului grafic clasic | 9  |
| 2.2.4 Soluții embedded pentru grafică 3D pe FPGA                | 10 |
| 2.2.5 Poziționarea proiectului față de soluțiile comerciale     | 10 |
| 2.3 Stadiul actual din punct de vedere științific               | 11 |
| 2.3.1 Pipeline grafic 3D complet pe FPGA                        | 11 |
| 2.3.2 Transformări afine 3D cu aritmetică fixed-point pe FPGA   | 12 |
| 2.3.3 GPU educațional pentru randare 2D pe FPGA                 | 12 |
| 2.3.4 Comparatie cu proiectul de față                           | 13 |
| 3 Simularea preciziei calculului efectuate                      | 14 |
| 3.1 Pierderea de precizie în reprezentarea cu virgulă fixă      | 14 |
| 3.2 Scopul simulării                                            | 14 |
| 3.3 Modulul DUT – Vertex Processing Unit (VPU)                  | 15 |
| 3.3.1 Interfața modulului                                       | 15 |
| 3.3.2 Arhitectura internă și fluxul de date                     | 16 |
| 3.3.3 Mașina de stări finite (FSM)                              | 16 |
| 3.3.4 Gestionarea overflow-ului                                 | 16 |
| 3.4 Componentele simulării                                      | 17 |
| 3.4.1 Configurare și parametri                                  | 17 |
| 3.4.2 Arhitectura testbench-ului                                | 18 |
| 3.4.3 Task-urile aferente                                       | 19 |
| 3.4.4 Raportarea rezultatelor                                   | 21 |
| 4 Concluzii                                                     | 22 |
| 4.1 Rezultate experimentale                                     | 22 |
| 4.1.1 Analiza preciziei în funcție de domeniul de intrare       | 23 |
| 4.1.2 Interpretarea rezultatelor                                | 24 |
| 4.2 Posibile îmbunătățiri                                       | 25 |
| 5 Codul sursă al proiectului                                    | 26 |
| 5.1 vpu_q.v                                                     | 26 |

|     |                        |    |
|-----|------------------------|----|
| 5.2 | tb_vpu_q.v . . . . .   | 29 |
|     | Bibliografie . . . . . | 34 |

|      |   |                                                   |
|------|---|---------------------------------------------------|
| BRAM | - | Block Random Access Memory                        |
| CLB  | - | Configurable Logic Block                          |
| DSP  | - | Digital Signal Processor                          |
| DUT  | - | Device Under Test                                 |
| FPGA | - | Field Programmable Gate Array                     |
| FP   | - | Floating Point                                    |
| FRAC | - | Fractional part (parte fracționară)               |
| FSM  | - | Finite State Machine                              |
| GPU  | - | Graphics Processing Unit                          |
| HDL  | - | Hardware Description Language                     |
| HDMI | - | High-Definition Multimedia Interface              |
| IEEE | - | Institute of Electrical and Electronics Engineers |
| INT  | - | Integer part (parte întreagă)                     |
| LUT  | - | Look-Up Table                                     |
| MAC  | - | Multiply-Accumulate                               |
| NDC  | - | Normalized Device Coordinates                     |
| Q    | - | Q-format                                          |
| RTL  | - | Register Transfer Level                           |
| SIMD | - | Single Instruction, Multiple Data                 |
| TMDS | - | Transition-Minimized Differential Signaling       |
| VGA  | - | Video Graphics Array                              |
| VPU  | - | Vertex Processing Unit                            |

# 1 INTRODUCERE

## 1.1 TEMA PROIECTULUI

Proiectul își propune proiectarea și implementarea unui accelerator grafic 3D pe o placă de dezvoltare FPGA, capabil să proceseze și să afișeze în timp real geometrie tridimensională pe un ecran. Sistemul realizează întregul flux de procesare grafică (de la coordonatele brute ale unui vertex în spațiul 3D până la pixelul afișat pe ecran) exclusiv în hardware dedicat, fără intervenția unui procesor de uz general.

Nucleul sistemului este un pipeline grafic structurat în trei etape succesive: **rotație, proiecție perspectivă și mapare în spațiul ecranului**. Fiecare etapă este implementată ca un submodul aritmetic independent, coordonat de o unitate centrală de procesare a vertex-urilor (*Vertex Processing Unit* – VPU), care orchestrează fluxul de date printr-o mașină de stări finite.

Toate calculele sunt efectuate în aritmetică cu **virgulă fixă semnată**, în formatul Q16.16, alegere motivată de simplitatea căii de date în hardware și de latența redusă față de echivalentul în virgulă mobilă. Formatul ales oferă o gamă reprezentabilă de  $\pm 32768$  unități, cu o rezoluție de aproximativ 15 micrometri, suficientă pentru scenariile grafice vizate.

O componentă distinctă a proiectului o reprezintă **analiza cantitativă a preciziei numerice**: prin simulare, rezultatele pipeline-ului în virgulă fixă sunt comparate sistematic cu un model de referință în virgulă mobilă dublă precizie, cuantificând eroarea introdusă de reprezentarea Q16.16 pe parcursul celor trei etape de procesare.

Proiectul acoperă astfel atât aspecte de *proiectare hardware digitală* (arhitectură pipeline, FSM, aritmetică în virgulă fixă, verificare funcțională), cât și aspecte de *grafică 3D* (transformări geometrice, proiecție perspectivă, sisteme de coordonate), constituind o punte între cele două domenii.

## 1.2 OBIECTIVELE PRINCIPALE

### 1.2.1 Obiective hardware

- **Afișare stabilă la 60 FPS** – sistemul trebuie să genereze un semnal video valid și stabil pentru un monitor standard, acoperind întreaga rezoluție a ecranului, fără artefacte sau întreruperi de semnal.
- **Redare în timp real a unui model 3D rotativ** – pipeline-ul grafic trebuie să proceseze și să afișeze un model tridimensional în mișcare continuă de rotație, demonstrând funcționarea corectă a tuturor celor trei etape: rotație, proiecție perspectivă și mapare pe ecran.

- **Precizie numerică acceptabilă** – eroarea introdusă de reprezentarea în virgulă fixă Q16.16 față de un model de referință în virgulă mobilă să nu depășească un prag de **5 pixeli** pentru coordonatele finale de ecran, asigurând o calitate vizuală adecvată.
- **Arhitectură complet parametrizabilă** – toate submodulele componente ale pipeline-ului trebuie să fie parametrizabile prin INT\_BITS și FRAC\_BITS, permițând reconfigurarea formatului numeric fără modificări structurale ale codului.

### 1.2.2 Obiective de documentație

- **Înțelegerea și documentarea pipeline-ului grafic 3D** – analiza și descrierea detaliată a etapelor unui pipeline grafic utilizat în industrie, corelând principiile teoretice cu implementarea hardware realizată.
- **Documentarea detaliată a submodulelor** – fiecare componentă a sistemului să fie însoțită de documentație completă, acoperind interfața de semnale, arhitectura internă, fluxul de date și comportamentul în cazuri limită (overflow, valori extreme). Codul Verilog asociat trebuie să fie integral comentat, facilitând înțelegerea și reutilizarea ulterioară.
- **Înțelegerea protocolului HDMI** – studiul principiilor de funcționare ale protocolului HDMI, cu accent pe mecanismul de transmitere a datelor video (codificare TMDS, sincronizare orizontală și verticală, generarea semnalului de ceas pixel) și integrarea acestuia cu pipeline-ul grafic implementat pe FPGA.

## 1.3 MOTIVAȚIE

Proiectul a pornit din fascinația pentru grafica 3D și curiozitatea de a înțelege mecanismele din spatele ei. De asemenea, provocarea de a construi un motor de randare în Verilog, fără a recurge la un procesor sau la biblioteci grafice, a transformat acest proiect dintr-o temă academică într-un exercițiu personal de înțelegere profundă a intersecției dintre grafică 3D și proiectarea hardware digitală.

## 2 STUDIU ASUPRA TEMEI ALESE

### 2.1 REPREZENTAREA ÎN VIRGULĂ FIXĂ

Pentru proiectul acesta, am avut nevoie să reprezint coordonatele punctelor 3D ca numere reale în hardware. În acest scop, am avut de ales între două metode de reprezentare:

- reprezentarea *fixed-point* (virgulă fixă) – folosește un număr fix de biți pentru partea întreagă și fracțională, cu cale de date simplă și latență redusă;
- reprezentarea *floating-point* (virgulă mobilă) – folosește notația științifică (exponent + mantisă), oferind o gamă dinamică mai largă, dar cu cost semnificativ mai mare de resurse logice și latență mai ridicată pe FPGA [1].

Prioritizând viteza de procesare și simplitatea implementării hardware, am ales reprezentarea în virgulă fixă. Pentru un număr  $I$  de biți alocați părții întregi și  $F$  biți pentru partea fracțională, un număr fixed-point este reprezentat pe  $I + F$  biți:

- biții  $[I + F - 1 : F]$  – partea întreagă
- biții  $[F - 1 : 0]$  – partea fracțională

Pentru implementare am ales formatul **Q16.16**, cu 16 biți pentru partea întreagă și 16 biți pentru partea fracțională. Această alegere oferă un echilibru potrivit pentru contextul graficii 3D: o gamă suficient de vastă pentru coordonate spațiale mari (până la  $\pm 32768$  m, acoperind scenarii de la  $-32000$  m la  $+32000$  m), cât și o rezoluție fină de  $2^{-16} \approx 0.000015$  m (aproximativ 15 micrometri), adecvată pentru reprezentarea precisă a geometriei la scară mică.

**Observație:** Notația  $Q/I.F$  nu impune o convenție unică în privința bitului de semn – acesta poate fi considerat implicit (exterior câmpului  $I$ ) sau inclus în numărul de biți alocați părții întregi, în funcție de implementare [1]. În lucrarea de față am ales **a doua convenție**: bitul de semn este inclus în cei 16 biți ai părții întregi, astfel încât din aceștia un bit reprezintă semnul, iar restul de 15 biți reprezintă mărimea, rezultând intervalul:

$$[-32768, 32767.99998474121]$$

Valorile extreme sunt definite în Verilog astfel:

```
localparam [WIDTH-1:0] MAX = {1'b0, {(WIDTH-1){1'b1}}};
localparam [WIDTH-1:0] MIN = {1'b1, {(WIDTH-1){1'b0}}};
```

unde  $WIDTH = INT\_BITS + FRAC\_BITS$ , deci:

$$MAX = 0x7FFF\_FFFF \quad (32767.99998474121)$$

$$MIN = 0x8000\_0000 \quad (-32768)$$

Rezoluția formatului, adică cea mai mică variație reprezentabilă între două valori consecutive, este:

$$RES = 2^{-16} = 0.0000152587890625$$

## 2.2 STADIUL ACTUAL DIN PUNCT DE VEDERE COMERCIAL

### 2.2.1 Pipeline-ul grafic în GPU-urile moderne

Unitatea de procesare grafică (*Graphics Processing Unit* – GPU) reprezintă soluția dominantă în industrie pentru accelerarea graficii 3D în timp real. Deși principiul de funcționare al unui GPU modern derivă din același pipeline conceptual (transformări geometrice, proiecție perspectivă și rasterizare) implementarea sa diferă fundamental de abordarea prezentată în această lucrare, atât prin arhitectură, cât și prin obiective.

Un GPU modern, precum cele din seria NVIDIA GeForce RTX sau AMD Radeon RX, este construit în jurul a mii de nuclee de procesare care execută în paralel operații pe vertex-uri și fragmente. Pipeline-ul grafic este complet programabil prin *shadere*: programe scrise în GLSL sau HLSL care rulează pe fiecare nucleu în parte, permițând efecte vizuale arbitrar de complexe: iluminare fizică, umbre volumetrice, ray tracing în timp real. Toate calculele sunt efectuate în virgulă mobilă pe 32 sau 16 biți (FP32/FP16), asigurând precizie ridicată indiferent de magnitudinea valorilor.

Prețul acestei flexibilități este consumul ridicat de putere (150–450W pentru GPU-uri de performanță) și costul de fabricație asociat miliardelor de tranzistoare integrate. Aceste caracteristici fac GPU-urile inadecvate pentru sisteme embedded cu constrângeri stricte de putere sau cost, deschizând spațiul pentru soluții alternative bazate pe FPGA.



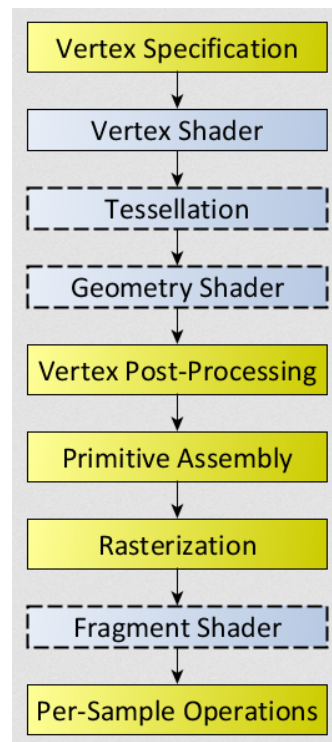


Figura 1: Pipeline-ul grafic programabil al unui GPU modern (sursă: documentație Khronos OpenGL [2])

### 2.2.2 FPGA-urile ca platformă pentru grafică 3D

Dispozitivele FPGA (*Field-Programmable Gate Array*) oferă o alternativă viabilă GPU-urilor în contextele în care flexibilitatea de programare și consumul redus de putere primează asupra performanței brute. Spre deosebire de GPU-uri, un FPGA implementează logica de procesare direct în hardware reconfigurabil, fără a recurge la un model de execuție bazat pe nuclee generice.

Producătorii majori de FPGA-uri – **AMD/Xilinx** și **Intel/Altera** – oferă familii de dispozitive cu resurse logice suficiente pentru implementarea unui pipeline grafic complet. Familia **Xilinx Artix-7** (utilizată frecvent în proiecte academice și embedded) conține până la 215.000 de celule logice și blocuri DSP dedicate înmulțirilor, potrivite pentru aritmetica fixed-point din etapele de rotație și proiecție [3]. La capătul superior al gamei, familia **Xilinx Ultrascale+** este utilizată în sisteme de procesare video industriale și aplicații de viziune artificială în timp real.

Un avantaj esențial al FPGA-urilor față de GPU-uri în contextul graficii 3D simple este **latența deterministă**: fiecare operație ocupă un număr fix și cunoscut de cicluri de ceas, fără variații cauzate de schedulare sau cache miss-uri. Aceasta face FPGA-urile potrivite pentru sisteme cu cerințe stricte de timp real, cum ar fi generatoarele de semnal video sincronizate cu un monitor.

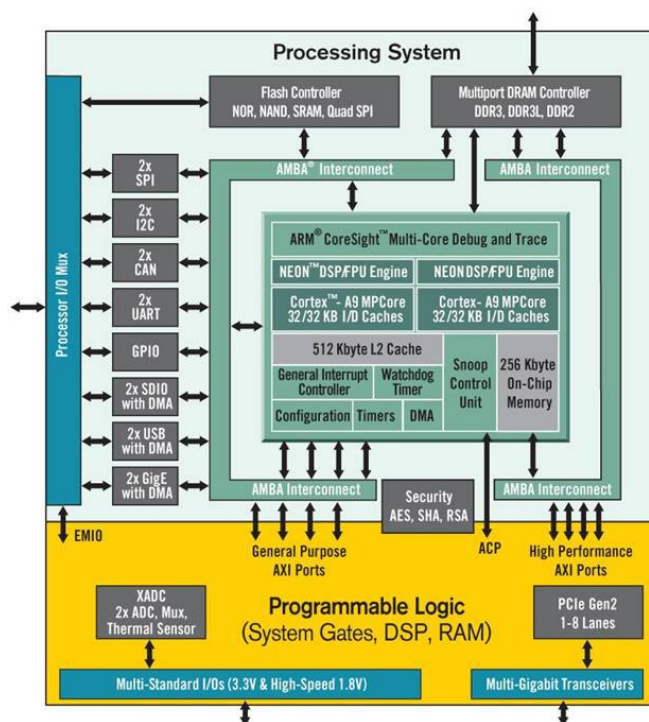


Figura 2: Arhitectura internă a unui FPGA – blocuri logice configurabile (CLB), blocuri DSP și memorii BRAM [3]

### 2.2.3 MiSTer FPGA – reimplementarea hardware-ului grafic clasic

Unul dintre cele mai relevante proiecte open-source care demonstrează capacitățile grafice ale FPGA-urilor moderne este platforma **MiSTer FPGA** [4], construită în jurul plăcii de dezvoltare **Terasic DE10-Nano** (Intel Cyclone V FPGA). MiSTer reimplementează în hardware reconfigurabil arhitecturi complete de calculatoare și console clasice – inclusiv unitățile lor de procesare grafică – demonstrând că un FPGA modern este capabil să reproducă fidel comportamentul hardware-ului grafic dedicat din generații anterioare.



Figura 3: Platforma MiSTer FPGA: DE10-Nano cu expansiuni pentru ieșire video și memorie

Din perspectiva proiectului de față, MiSTer este relevant deoarece confirmă că generarea unui semnal video stabil la 60 FPS, sincronizat cu un monitor modern prin HDMI, este realizabilă pe un FPGA de complexitate medie, fără resurse dedicate de procesare grafică. Nucleele MiSTer implementează generatoare de semnal video în Verilog/SystemVerilog, folosind tehnici similare celor adoptate în acest proiect: temporizare pe bază de contor de pixeli, generare de semnal HSYNC/VSNC și transmisie serializată prin TMDS.

#### 2.2.4 Soluții embedded pentru grafică 3D pe FPGA

În afara platformelor de tip hobbyist, există soluții comerciale care integrează procesare grafică 3D simplă pe FPGA în sisteme embedded industriale și educaționale.

**SpinalHDL GPU** și proiecte similare open-source demonstrează că un rasterizator software cu pipeline fix poate fi implementat pe FPGA-uri de clasă medie (Xilinx Spartan-7, Intel MAX 10), producând imagini 3D la rezoluții de  $320 \times 240$  sau  $640 \times 480$  pixeli [5]. Aceste implementări folosesc în general aritmetică fixed-point pentru calcule geometrice și virgulă mobilă doar pentru etapele de iluminare, urmând același raționament adoptat în această lucrare: simplitatea și viteza căii de date fixed-point depășesc dezavantajul pierderii de precizie pentru geometrie simplă.

**Soft-core GPU-urile** reprezintă o categorie aparte: procesoare grafice implementate integral în logica reconfigurabilă a unui FPGA, fără componente hardware dedicate. Proiecte precum **f32c** sau **MCPU** oferă un nucleu RISC cu accelerare grafică primitivă, capabil să genereze output VGA/HDMI direct din FPGA. Față de aceste abordări, proiectul de față se diferențiază prin implementarea unui pipeline grafic pur hardware, fără procesor și fără cod software, ceea ce garantează latența minimă și comportamentul determinist.

#### 2.2.5 Poziționarea proiectului față de soluțiile comerciale

Proiectul de față nu urmărește să concureze cu GPU-urile comerciale sau cu platformele grafice complexe, ci să demonstreze că un pipeline grafic 3D funcțional poate fi implementat integral în hardware digital, folosind exclusiv concepte de electronică digitală sincronă. Față de soluțiile prezentate, contribuțiile distinctive ale proiectului sunt:

- **Pipeline complet parametrizabil** – formatul numeric ( $Q/I.F$ ) este configurabil la sinteză, permițând adaptarea compromisului precizie/domeniu fără modificări structurale;
- **Verificare cantitativă a preciziei** – spre deosebire de majoritatea implementărilor open-source care nu includ o analiză formală a erorii numerice, proiectul de față include o campanie de simulare sistematică cu model de referință în virgulă mobilă;

- **Arhitectură modulară și documentată** – fiecare submodul este proiectat independent și documentat detaliat, facilitând extinderea sau reutilizarea în alte proiecte.

## 2.3 STADIUL ACTUAL DIN PUNCT DE VEDERE ȘTIINȚIFIC

Implementarea unui pipeline grafic 3D în hardware reconfigurabil reprezintă un subiect activ de cercetare, abordat din perspective diferite în funcție de obiectivele urmărite: performanță maximă, consum redus de putere, simplitate educațională sau acuratețe numerică. Lucrările prezentate în continuare sunt relevante direct pentru principiile constructive adoptate în proiectul de față.

### 2.3.1 Pipeline grafic 3D complet pe FPGA

Lucrarea [6] propune o implementare hardware completă a pipeline-ului grafic 3D pe FPGA, compatibilă cu standardele OpenGL ES 2.0 și Shader Model 3.0. Arhitectura acoperă întregul flux de procesare grafică: *vertex shader*, motor de clipping, motor de configurare a triunghiurilor (*triangle setup engine*), rasterizator, pixel shader și operator raster (*raster operator*).

Din perspectiva proiectului de față, cea mai relevantă componentă este **vertex shader-ul**, implementat ca o structură SIMD pe 128 de biți cu patru fire de execuție concurente, folosind aritmetică în virgulă mobilă pe 32 de biți. Aceasta contrastează direct cu abordarea adoptată în această lucrare – pipeline serial cu aritmetică fixed-point Q16.16 – ilustrând compromisul fundamental dintre flexibilitate/precizie (virgulă mobilă, SIMD) și simplitate/latență deterministă (virgulă fixă, pipeline secvențial).

Implementarea a fost realizată pe un FPGA Xilinx Virtex-5, operând la 35 MHz, și a demonstrat capacitatea de a rasteriza 70.000 de triunghiuri din modelul Stanford Bunny la 30 cadre/secundă. Complexitatea totală a depășit 1.486 milioane de porți logice echivalente, cu o memorie internă de 57 KB și un framebuffer extern de 2.4 MB – dimensiuni care ilustrează costul resurselor unui pipeline grafic complet față de soluția minimalistă propusă în această lucrare, care nu include rasterizare sau shading.

Tabel 1: Resursele logice ale pipeline-ului grafic propus de Kim et al. [6]

| Modul           | Logică        | Memorie               |
|-----------------|---------------|-----------------------|
| Vertex Shader   | 845K          | 23 KB                 |
| Clipping Engine | 80K           | 3 KB                  |
| Triangle Setup  | 106K          | 2 KB                  |
| Rasterizer      | 50K           | 2 KB                  |
| Pixel Shader    | 370K          | 27 KB                 |
| ROP             | 35K           | 2.4 MB (extern)       |
| <b>Total</b>    | <b>1.486M</b> | <b>57 KB + 2.4 MB</b> |

### 2.3.2 Transformări afine 3D cu aritmetică fixed-point pe FPGA

În [7] se investighează utilizarea FPGA-urilor ca acceleratoare de cost redus pentru transformările afine 3D: rotație, scalare, forfecare și translație – prin înmulțire matriceală paralelă în aritmetică fixed-point. Lucrarea este direct relevantă pentru etapa de rotație din pipeline-ul proiectului de față, care realizează exact același tip de transformare afină.

Transformările afine sunt exprimate în coordonate omogene, permițând reprezentarea uniformă a tuturor operațiilor (inclusiv translația) ca înmulțiri matriceale de forma  $NP = T \times OP$ , unde  $T$  este matricea de transformare  $4 \times 4$ ,  $OP$  conține pozițiile vechi ale vertex-urilor și  $NP$  conține pozițiile noi. Matricele de rotație pentru axele X, Y și Z sunt cele standard:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_z = \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Arhitectura propusă constă dintr-un multiplicator matriceal paralel cu  $p$  elemente de procesare (PE) identice, fiecare conținând un acumulator multiply-accumulate (MAC) în fixed-point. Coordonatele vertex-urilor sunt reprezentate pe **22 de biți** (14 biți pentru partea fracționară), comparabil cu formatul Q16.16 (16 biți fracționari) ales în această lucrare. Autorii justifică alegerea fixed-point față de virgulă mobilă prin același argument adoptat și în proiectul de față: reprezentarea în virgulă mobilă are cerințe de resurse semnificativ mai mari și performanță mai scăzută pe dispozitivele FPGA disponibile [7].

Rezultatele raportate arată că implementarea bazată pe blocurile CoreGen Xilinx (multiplicator și sumator întregi cu semn în paralel) depășește în performanță varianta cu biblioteca fixed-point Celoxica, atingând viteze de până la 35 MHz pentru transformări pe obiecte cu până la  $2^{19}$  vertex-uri. Această observație este relevantă pentru proiectul de față, care folosește de asemenea blocuri DSP dedicate ale FPGA-ului pentru înmulțirile fixed-point din etapele de rotație și proiecție.

### 2.3.3 GPU educațional pentru randare 2D pe FPGA

O abordare cu scop educațional este prezentată în [8], unde se prezintă proiectarea și implementarea unui accelerator grafic 2D pe FPGA cu scop educațional, ținând plasa de dezvoltare Nexys 4 echipată cu FPGA Artix-7 și utilizând Verilog ca limbaj de descriere hardware. Deși obiectivul lucrării este randarea 2D, arhitectura propusă și metodologia de verificare sunt relevante pentru proiectul de față din mai multe perspective.

Arhitectura este structurată în opt submodule cu responsabilități clare: ROM (stocare vertex-uri), RAM, transfer ROM→RAM, algoritmul Bresenham pentru desenare de linii, umplere triunghiuri, buffer video, sincronizare VGA și un modul MASTER de control. Comunicarea între submodule este gestionată prin semnale de control și registre de stare: o abordare structural similară cu pipeline-ul FSM din modulul `vpw_q` al acestei lucrări, unde un modul central coordonează execuția secvențială a submodulelor aritmetice.

Un aspect important subliniat de autori este că arhitecturile GPU industriale sunt excesiv de complexe pentru scopuri educaționale, motivând o abordare minimalistă și ușor de înțeles. Proiectul de față urmărește același principiu, limitând pipeline-ul la etapele geometrice esențiale (rotație, proiecție, mapare) fără rasterizare sau shading, și documentând detaliat fiecare submodul pentru înțelegerea principiilor fundamentale.

### 2.3.4 Comparație cu proiectul de față

Tabelul 2 sintetizează principalele caracteristici ale lucrărilor analizate în raport cu proiectul de față.

Tabel 2: Comparație între lucrările analizate și proiectul de față

| Caracteristică      | Kim et al. [6] | Bensaali et al. [7] | Saha et al. [8] | Proiect de față    |
|---------------------|----------------|---------------------|-----------------|--------------------|
| Tip aritmetică      | Virgulă mobilă | Fixed-point 22b     | Întreg          | Fixed-point Q16.16 |
| Etape pipeline      | Complet (7)    | Transformări afine  | Rasterizare 2D  | Geometrie 3D (3)   |
| Ieșire              | VGA 640×480    | Fără afișaj         | VGA             | HDMI               |
| Frecvență           | 35 MHz         | până la 35 MHz      | –               | 125 MHz            |
| Verificare numerică | Nu             | Nu                  | Simulare        | Golden Model FP64  |
| Parametrizabil      | Nu             | Parțial             | Nu              | Da ( $Q/I.F$ )     |

Față de lucrările analizate, proiectul de față aduce ca element distinctiv **verificarea cantitativă sistematică a preciziei numerice** prin compararea directă a rezultatelor fixed-point cu un model de referință în virgulă mobilă dublă precizie, un aspect neglijat în toate cele trei lucrări citate, care fie nu includ o analiză a erorii numerice, fie se limitează la verificarea funcțională calitativă.

### 3 SIMULAREA PRECIZIEI CALCULELOR EFECTUATE

#### 3.1 PIERDEREA DE PRECIZIE ÎN REPREZENTAREA CU VIRGULĂ FIXĂ

Deși rezoluția de  $2^{-16}$  pare suficient de fină pentru coordonate individuale, în cadrul pipeline-ului VPU aceasta devine o sursă semnificativă de eroare. Motivul este că procesarea unui vertex presupune trei etape succesive de calcul (rotație, proiecție perspectivă și mapare pe ecran) fiecare introducând propria sa eroare de cuantizare, care se **acumulează** de-a lungul pipeline-ului.

Erorile de cuantizare apar din două mecanisme distincte:

- **Trunchiere la înmulțire** – produsul a două numere Q16.16 are  $2 \times 32 = 64$  biți, din care se păstrează doar 32 prin trunchiere, pierzând biții de precizie inferioară;
- **Overflow la valori mari** – dacă rezultatul unui calcul intermediar depășește intervalul  $[-32768, 32767]$ , acesta este saturat la MAX sau MIN, introducând o eroare brutală în coordonatele finale.

Proiecția perspectivă amplifică în mod special această problemă: formula  $x_p = x_r \cdot f / z_r$  implică o împărțire, iar pentru valori mici ale lui  $z_r$  (punct aproape de cameră), rezultatul poate depăși domeniul reprezentabil, provocând overflow. Totodată, coordonatele generate aleatoriu pe 16 biți întregi pot produce valori de ordinul zecilor de mii, care după înmulțire cu factorul focal depășesc cu ușurință limita superioară a formatului.

#### 3.2 SCOPUL SIMULĂRII

Plecând de la aceste observații, scopul simulării este dublu:

1. **Cuantificarea erorii acumulate** – prin compararea rezultatelor DUT-ului (calcul în virgulă fixă Q16.16) cu un model de referință (calcul în virgulă mobilă dublă precizie), se determină eroarea absolută introdusă de pipeline-ul fix-point, exprimată în pixeli pe ecranul final.
2. **Identificarea condițiilor de degradare** – prin varierea preciziei biților de intrare (`precision_*_int/frac`) și a pragului de toleranță `THRESHOLD`, se identifică domeniile de valori de intrare pentru care eroarea de cuantizare depășește un prag acceptabil, furnizând informații despre limitele practice ale formatului Q16.16 în contextul graficii 3D în timp real.

Un rezultat cu rată de erori ridicată nu indică neapărat o implementare greșită, ci reflectă limitele intrinseci ale reprezentării în virgulă fixă atunci când datele de intrare acoperă întregul domeniu al formatului Q16.16. Simularea permite astfel trasarea unui compromis clar între domeniul de valori acceptat la intrare și precizia coordonatelor de ecran la ieșire.

### 3.3 MODULUL DUT – VERTEX PROCESSING UNIT (VPU)

Modulul `vpu_q` reprezintă unitatea principală de procesare geometrică, având rolul de a transforma coordonatele unui punct din spațiul 3D în coordonate de ecran 2D. Procesarea este realizată în trei etape succesive – rotație, proiecție perspectivă și mapare în spațiul ecranului – fiecare implementată printr-un submodul aritmetic dedicat, controlat de o mașină de stări finite (FSM) de tip Moore.

Toate calculele sunt efectuate în reprezentare cu virgulă fixă semnată, parametrizabilă prin `INT_BITS` și `FRAC_BITS` (implicit Q16.16).

#### 3.3.1 Interfața modulului

Tabelul 3 prezintă semnalele de intrare și ieșire ale modulului.

Tabel 3: Semnalele modulului `vpu_q`

| Semnal                 | Direcție | Lățime | Descriere                                                         |
|------------------------|----------|--------|-------------------------------------------------------------------|
| <code>clk</code>       | Input    | 1      | Semnal de ceas                                                    |
| <code>rst_n</code>     | Input    | 1      | Reset asincron, activ în 0                                        |
| <code>start</code>     | Input    | 1      | Puls de inițiere a procesării unui nou punct                      |
| <code>rotation</code>  | Input    | 3      | Selectează tipul de rotație (X, Y, Z sau combinat)                |
| <code>angle</code>     | Input    | 10     | Unghi de rotație (în unități de tabel LUT)                        |
| <code>x, y, z</code>   | Input    | WIDTH  | Coordonatele punctului în spațiul 3D                              |
| <code>f</code>         | Input    | WIDTH  | Distanța focală pentru proiecție perspectivă                      |
| <code>w, h</code>      | Input    | WIDTH  | Dimensiunile ecranului (lățime, înălțime)                         |
| <code>cam_z</code>     | Input    | WIDTH  | Translația camerei pe axa Z                                       |
| <code>xs, ys</code>    | Output   | WIDTH  | Coordonatele finale în spațiul ecranului                          |
| <code>valid</code>     | Output   | 1      | Ridicat în starea DONE; indică că <code>xs, ys</code> sunt valide |
| <code>overflow</code>  | Output   | 1      | OR logic al overflow-urilor din toate etapele                     |
| <code>dbg_state</code> | Output   | 4      | Starea curentă a FSM-ului (pentru depanare)                       |

unde  $WIDTH = INT\_BITS + FRAC\_BITS$ .



### 3.3.2 Arhitectura internă și fluxul de date

Modulul este structurat ca un **pipeline secvențial** (un singur punct activ la un moment dat), controlat de FSM. Figura 4 ilustrează fluxul datelor între registrele interne și submodulele aritmetice.

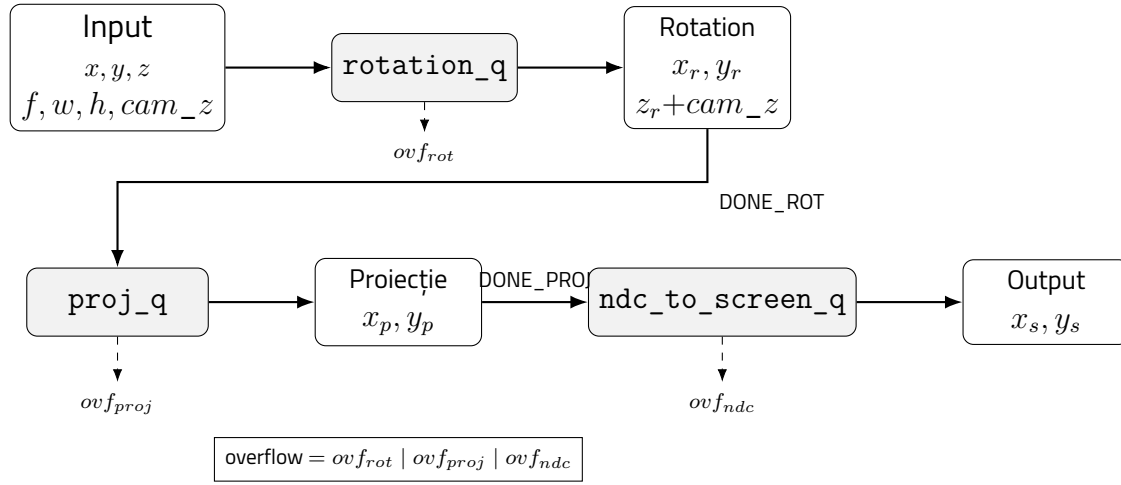


Figura 4: Fluxul de date în modulul vpu\_q

Un detaliu important: în starea DONE\_ROT, coordonata  $z_r$  este augmentată cu  $cam\_z$  înainte de a fi transmisă submoduleului de proiecție, realizând astfel translația camerei pe axa Z fără un submodul suplimentar.

### 3.3.3 Mașina de stări finite (FSM)

FSM-ul are 11 stări, codificate pe 4 biți. Structura este de tip Moore: ieșirile (pulsurile  $rot\_start$ ,  $proj\_start$ ,  $ndc\_start$ ,  $valid$ ) depind exclusiv de starea curentă.

Fiecare submodul aritmetic este activat printr-un puls de **un singur ciclu** (stările  $START\_*$ ), iar FSM-ul rămâne în starea  $CALC\_*$  până când submodulul ridică semnalul  $valid$ .

### 3.3.4 Gestionarea overflow-ului

Fiecare submodul aritmetic expune un semnal **overflow** propriu, ridicat atunci când rezultatul unui calcul intermediar depășește domeniul reprezentabil în formatul Q16.16. Modulul vpu\_q acumulează aceste semnale în registre intermediare ( $ovf\_rot\_r$ ,  $ovf\_proj\_r$ ) și le combină în starea DONE prin operația:

$$overflow = ovf\_rot\_r \mid ovf\_proj\_r \mid ovf\_ndc\_w$$

Semnalul de overflow rămâne valid pe durata unui ciclu de ceas (starea DONE) și poate fi folosit de un controller extern pentru a invalida rezultatul curent.

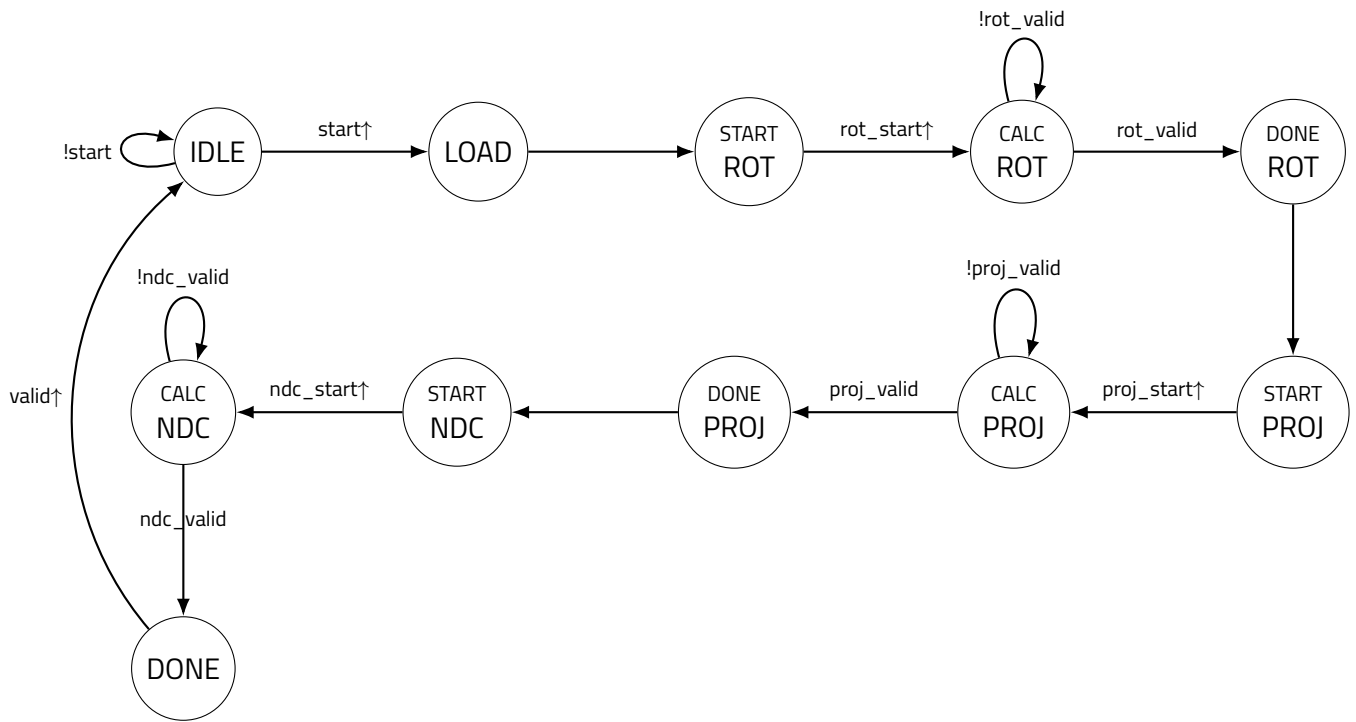


Figura 5: FSM-ul unității `vpu_q` – toate tranzițiile, inclusiv self-loop-urile de așteptare

### 3.4 COMPONENTELE SIMULĂRII

Testbench-ul `tb_vpu_q` verifică funcționalitatea modului `vpu_q` prin compararea rezultatelor sale cu un *model de referință* (Golden Model) implementat în aritmetică `rea1` (virgulă mobilă dublă precizie). Strategia de verificare este de tip **self-checking**: fiecare test determină automat dacă rezultatul DUT-ului se încadrează într-un prag de toleranță predefinit, fără intervenție manuală.

#### 3.4.1 Configurare și parametri

Tabelul 4 prezintă parametrii de configurare ai testbench-ului, grupați în două categorii: parametrii fixați ai platformei (care nu se modifică) și parametrii ajustabili ai sesiunii de testare.

Parametrul `THRESHOLD` este central pentru strategia de verificare: o diferență de cel mult 0.5 pixeli față de Golden Model este considerată acceptabilă, reflectând eroarea de cuantizare inerentă reprezentării `Q16.16` ( $RES = 2^{-16} \approx 0.000015$ ). Cazurile cu `overflow` activ sunt excluse din calculul erorilor și contorizate separat.

Tabel 4: Parametrii testbench-ului tb\_vpu\_q

| Parametru                             | Valoare implicită | Descriere                                                              |
|---------------------------------------|-------------------|------------------------------------------------------------------------|
| <i>Parametri fixați ai platformei</i> |                   |                                                                        |
| PER                                   | 8 ns              | Perioada ceasului (125 MHz)                                            |
| INT_BITS                              | 16                | Biți pentru partea întreagă (include semnul)                           |
| FRAC_BITS                             | 16                | Biți pentru partea fracționară                                         |
| WIDTH                                 | 32                | Lățimea totală a cuvântului (INT+FRAC)                                 |
| SCALE                                 | $2^{16}$          | Factor de scalare Q16.16 pentru conversia real fixed                   |
| MAX_R                                 | 0x7111_1111       | Valoare de saturație superioară (overflow)                             |
| MIN_R                                 | 0x8000_0000       | Valoare de saturație inferioară (overflow)                             |
| <i>Parametri ajustabili</i>           |                   |                                                                        |
| NUM_RAND                              | 2000              | Numărul de teste aleatorii per campanie                                |
| THRESHOLD                             | 0.5 px            | Toleranța maximă acceptată (în pixeli)                                 |
| CAMERA_Z                              | 1.5               | Distanța camerei față de origine pe axa Z                              |
| FOCAL_LENGTH                          | 0.05              | Distanța focală (echivalent $\approx 50$ mm)                           |
| SCREEN_WIDTH                          | 320 px            | Lățimea ecranului de referință                                         |
| SCREEN_HEIGHT                         | 240 px            | Înălțimea ecranului de referință                                       |
| precision*_int/frac                   | 16/16             | Numărul de biți de precizie pentru generarea aleatorie a coordonatelor |

### 3.4.2 Arhitectura testbench-ului

Testbench-ul este structurat în cinci task-uri cu responsabilități clare, apelate dintr-o subrutină `initial` principală. Figura 6 ilustrează ierarhia acestora și fluxul de control.

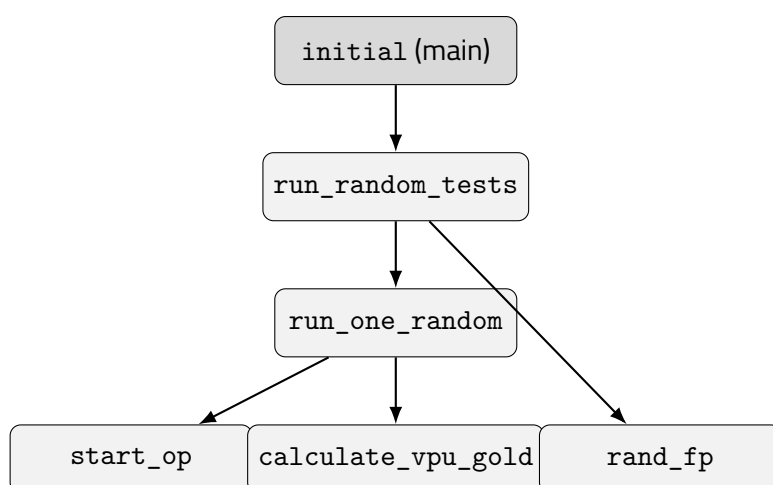


Figura 6: Ierarhia task-urilor din tb\_vpu\_q

### 3.4.3 Task-urile aferente

**Task start\_op** Aplică stimulii la intrările DUT-ului și generează pulsul de start de un singur ciclu de ceas, conform protocolului de interfață al modulului vpu\_q. Secvența exactă este:

1. se încarcă toate datele de intrare în registrele testbench-ului;
2.  $start \leftarrow 1$  pentru un ciclu de ceas;
3.  $start \leftarrow 0$  (DUT-ul preia datele și începe procesarea).

**Task calculate\_vpu\_gold** Implementează modelul de referință al întregului pipeline VPU în aritmetică real, reproducând fidel cele trei etape ale DUT-ului:

- **Rotație:** unghiul este reprezentat pe 10 biți în format Q9.1 nesemnat, cu rezoluție de  $0.5^\circ$ , acoperind domeniul  $[0^\circ, 359.5^\circ]$  prin valori întregi în intervalul  $[0, 719]$ . Conversia în radiani se realizează prin:

$$\theta = \frac{ang}{2} \cdot \frac{\pi}{180}$$

unde împărțirea la 2 decodifică reprezentarea Q9.1 (recuperează valoarea reală în grade), iar înmulțirea cu  $\pi/180$  convertește în radiani. După obținerea unghiului, se aplică una dintre cele 6 matrice de rotație parțiale, selectată prin semnalul `rotation`.

- **Translație cameră:**  $z_r \leftarrow z_r + cam\_z$ , identic cu operația din starea `DONE_ROT` a DUT-ului.
- **Proiecție perspectivă:**  $x_p = x_r \cdot f/z_r$ ,  $y_p = y_r \cdot f/z_r$ , cu tratare explicită a cazului  $z_r = 0$  (saturare la `MAX_R` sau `MIN_R`).
- **Mapare pe ecran:**  $x_s = (x_p \cdot h + w)/2$ ,  $y_s = (h - y_p \cdot h)/2$ , cu inversarea axei Y specific spațiului ecran.

**Task rand\_fp** Generează o valoare fixed-point semnată aleatorie, respectând o precizie configurabilă prin `precision*_int/frac`. Aceasta permite restrângerea domeniului de intrare (de exemplu, coordonate cu parte întreagă pe 8 biți în loc de 16) pentru a studia comportamentul DUT-ului la valori mici sau pentru a evita saturarea prematură.

Valoarea brută este generată cu `$random`, truncată la domeniul  $[-2^{int+frac-1}, 2^{int+frac-1})$ , apoi aliniată la formatul Q16.16 prin deplasare la stânga cu  $FRAC\_BITS - rand\_frac\_bits$ .

**Task** `run_one_random` Orchestrează un singur test complet, urmând pașii din Figura 7:

1. Convertește valorile fixed-point de intrare în `real` (împărțire la `SCALE`) pentru a le transmite modelului de referință.
2. Calculează rezultatele așteptate cu `calculate_vpu_gold`.
3. Convertește înapoi rezultatele așteptate în fixed-point (`$rtoi(val * SCALE)`) pentru comparație directă cu DUT-ul.
4. Stimulează DUT-ul prin `start_op` și așteaptă ridicarea semnalului `valid`.
5. Calculează erorile absolute  $err_x = (xs_{DUT} - xs_{gold})/SCALE$  și  $err_y$  (în pixeli).
6. Clasifică rezultatul: **SKIP** (overflow detectat), **EROARE** (depășire prag), sau **OK**.

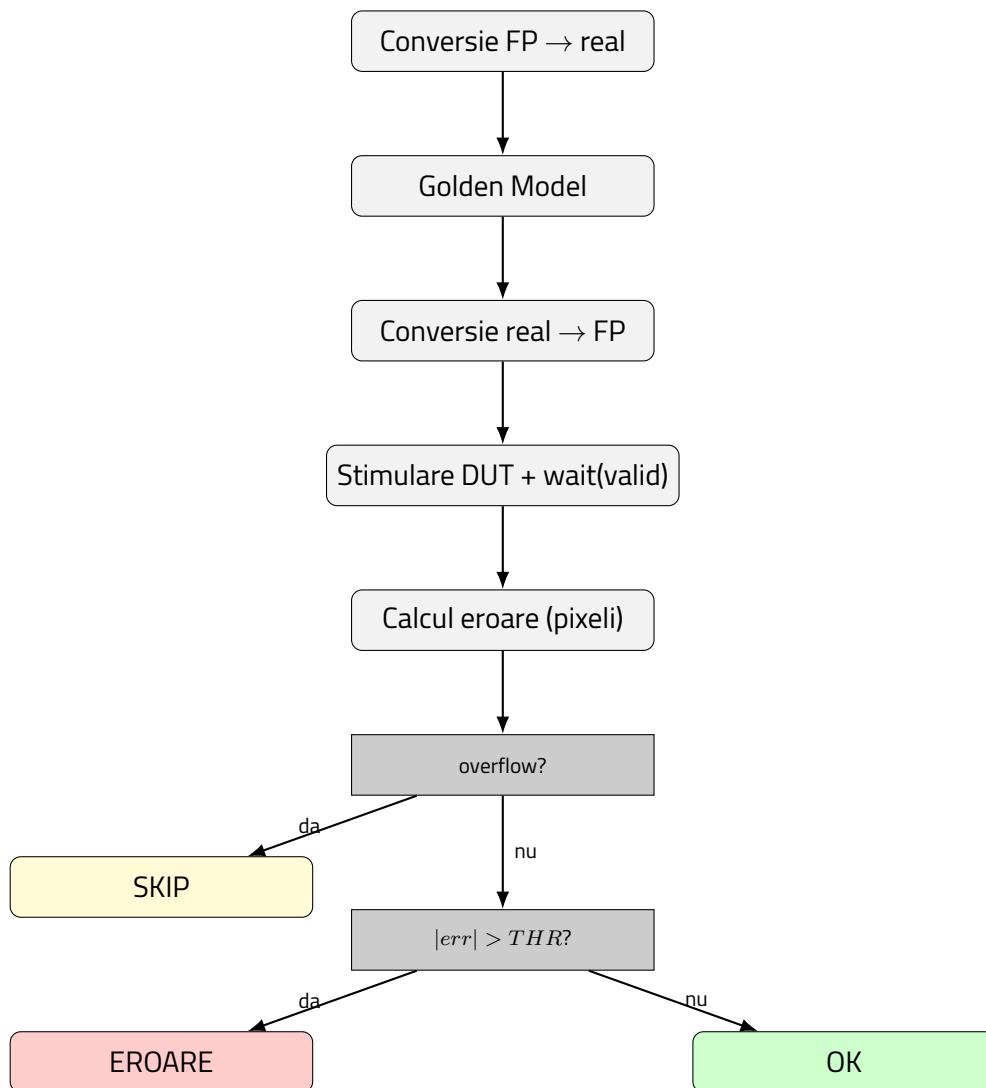


Figura 7: Fluxul de execuție al unui test singular în `run_one_random`

**Task run\_random\_tests** Execută testele prin apelarea repetată a `run_one_random` de `NUM_RANDOM` ori, cu parametrii geometrici ai ecranului fixați (`SCREEN_WIDTH`, `SCREEN_HEIGHT`, `FOCAL_LENGTH`) și coordonatele vertexului generate aleatoriu la fiecare iterație. Tipul de rotație este ales uniform din  $\{0, 1, 2, 3, 4, 5\}$ , iar unghiul din domeniul numere întregi  $[0, 719]$ .

### 3.4.4 Raportarea rezultatelor

La finalul testării, `testbench`-ul exportează un sumar complet al execuției într-un fișier text (`rezultate.txt`) deschis la începutul simulării prin `$fopen` și închis explicit înainte de `$finish`. Dacă fișierul nu poate fi creat, simularea se oprește cu mesaj de eroare. Același sumar este afișat și în consola simulatorului prin `$display`.

Raportul conține două secțiuni: **configurația platformei** și **statisticile sesiunii de testare**.

■ **Configurația platformei** – parametrii fixați ai rulării: frecvența ceasului, formatul numeric Q, distanța focală, rezoluția ecranului și precizia biților folosiți la generarea aleatorie a coordonatelor.

■ **Statisticile sesiunii de testare** – rezultatele agregate:

- *Teste totale* – numărul de cazuri rulate (`NUM_RANDOM`);
- *Erori* – cazuri în care  $|err_x| > THR$  sau  $|err_y| > THR$ , cu `THRESHOLD` exprimat în pixeli;
- *Overflow-uri* – cazuri excluse din verificare din cauza depășirii domeniului numeric Q16.16; acestea nu sunt incluse în numărătoarea erorilor;
- *Rata de erori (%)* – calculată conform formulei:

$$\text{Rata de erori} = \frac{\text{erori}}{\text{teste totale}} \times 100\%$$

Un exemplu de raport generat pentru o sesiune de 2000 de teste cu pragul de toleranță `THRESHOLD = 10` pixeli este prezentat mai jos:

-----

=== TEST VPU FINALIZAT ===

Frecventa ceas = 125 MHz

Format Q = Q16.16 semnat

Focala = 50.0031 mm

```

Latime ecran    = 320 px
Inaltime ecran  = 240 px
Biti random X   = 16 INT, 16 FRAC
Biti random Y   = 16 INT, 16 FRAC
Biti random Z   = 16 INT, 16 FRAC

```

```

Prag eroare     = 10 pixeli
Teste totale    = 2000
Erori           = 640
Overflow-uri    = 233
RATA DE ERORI   = 36.219581 %

```

-----

Din cele 2000 de teste, 233 au fost marcate cu overflow și excluse din evaluare, iar din restul de 1767 cazuri valide, 640 au depășit pragul de 10 pixeli, rezultând o rată de erori de 36.22%. Această valoare ridicată motivează analiza din secțiunea următoare, unde se investighează influența preciziei biților de intrare și a valorii pragului asupra acurateței reprezentării în format Q16.16.

## 4 CONCLUZII

### 4.1 REZULTATE EXPERIMENTALE

Figura 8 prezintă forma de undă capturată în simulatorul logic pentru un singur ciclu de procesare al modulului `vpu_q`.

Se poate observa că semnalul `start` este activ pentru un singur ciclu de ceas, după care FSM-ul parcurge secvențial toate cele 11 stări (vizibile pe semnalul `dbg_state`:  $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 10$ ), iar semnalul `valid` este ridicat la finalul procesării. Măsurând între frontul crescător al semnalului `start` și frontul crescător al semnalului `valid`, durata unui ciclu complet VPU este de exact **79 tacte de ceas**, ceea ce la frecvența de 125 MHz corespunde unei latențe de:

$$T_{VPU} = 79 \times \frac{1}{125 \times 10^6} = 632 \text{ ns}$$

per vertex procesat.

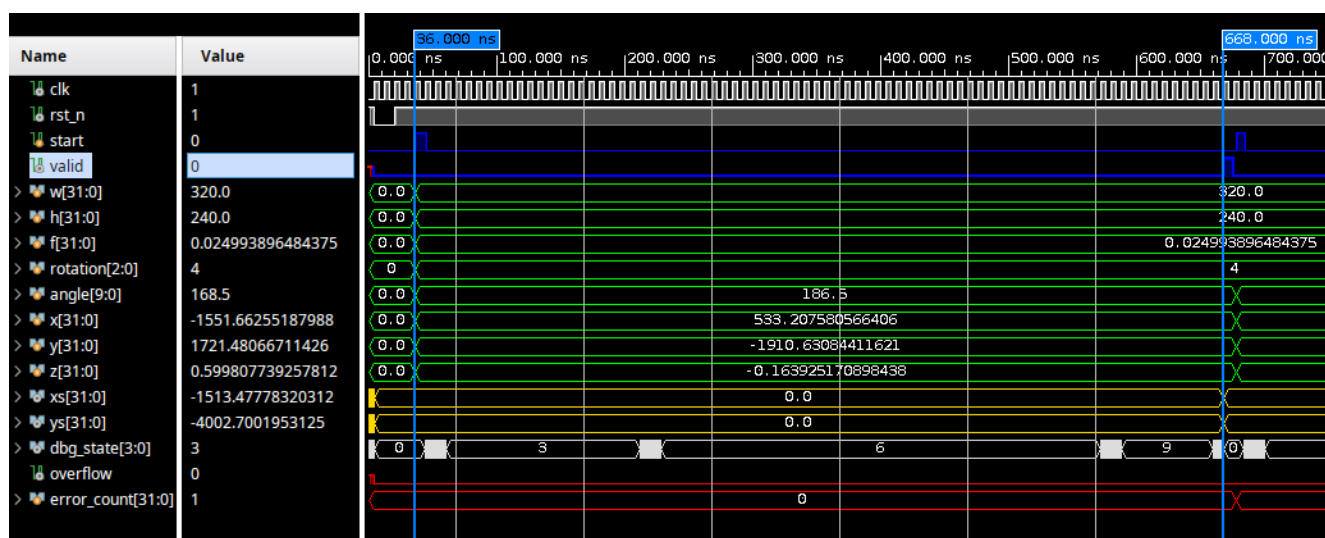


Figura 8: Forma de undă pentru un ciclu complet de procesare VPU (rotație + proiecție + mapare pe ecran)

#### 4.1.1 Analiza preciziei în funcție de domeniul de intrare

Sesiunea de testare a urmărit influența a doi factori asupra ratei de erori: **domeniul biților de intrare** (prin parametrii `precision_*_int`) și **distanța focală**  $f$ . Toate rulările au folosit 2000 de teste aleatorii cu pragul de toleranță `THRESHOLD = 2` pixeli. Rezultatele sunt sintetizate în Tabelul 5.

Tabel 5: Rezultatele sesiunii de simulare – rata de erori în funcție de domeniul de intrare și distanța focală

| INT | FRAC      | Focală (mm) | Erori | Overflow-uri | Rată erori (%) |
|-----|-----------|-------------|-------|--------------|----------------|
| 16  | 16        | 50.00       | 1676  | 233          | 94.85          |
| 16  | 16        | 25.00       | 1458  | 233          | 82.51          |
| 16  | 16        | 12.50       | 895   | 233          | 50.65          |
| 16  | 16        | 0.015       | 0     | 233          | 0.00           |
| 12  | 16        | 25.00       | 504   | 0            | 25.20          |
| 12  | 16 (Z: 4) | 25.00       | 362   | 21           | 18.29          |
| 8   | 16        | 50.00       | 4     | 0            | 0.20           |
| 8   | 16        | 25.00       | 3     | 0            | 0.15           |
| 4   | 16        | 50.00       | 0     | 1            | 0.00           |
| 4   | 16        | 25.00       | 1     | 0            | 0.05           |



#### 4.1.2 Interpretarea rezultatelor

Analiza rezultatelor din Tabelul 5 evidențiază două surse principale de eroare, independente de precizia aritmetică intrinsecă a formatului Q16.16.

**Absența verificării clip space** Principala cauză a ratei ridicate de erori este absența unui test de *clip space*: vertex-urile sunt procesate indiferent de poziția lor față de volumul de vizibilitate al camerei, producând coordonate de ecran în afara rezoluției fizice fără ca acest lucru să fie semnalat prin *overflow*. Influența distanței focale asupra ratei de erori confirmă indirect această ipoteză:  $f$  determină implicit câmpul vizual al camerei (FOV), astfel că o focală mai mare îngustează piramida de vizibilitate și exclude mai mulți vertex-uri din domeniul valid. La  $f = 50\text{mm}$ , rata de erori atinge 94.85%, în timp ce la  $f = 0.015\text{mm}$  scade la 0%, deoarece focala extrem de mică lărgeste FOV-ul până aproape de  $180^\circ$ , incluzând practic toți vertex-urile în volumul de vizibilitate și menținând coordonatele proiectate  $x_p = x_r \cdot f / z_r$  la valori reprezentabile în Q16.16.

**Overflow în etapa de rotație** Cele 233 de cazuri cu *overflow* apar consistent în toate rulările cu  $\text{INT} = 16$ , indiferent de distanța focală, indicând că sursa lor este etapa de **rotație**. Coordonatele cu parte întreagă pe 16 biți pot atinge valori de ordinul zecilor de mii, iar produsele intermediare din calculul matricei de rotație depășesc domeniul reprezentabil al formatului Q16.16 încă înainte de a ajunge la etapa de proiecție. Aceste cazuri sunt excluse din calculul ratei de erori, deoarece rezultatul DUT-ului este nedefinit în prezența *overflow*-ului.

## 4.2 POSIBILE ÎMBUNĂTĂȚIRI

**Verificarea clip space** Implementarea unui modul de *clipping* care să testeze, după etapa de proiecție, dacă coordonatele NDC ale unui vertex se încadrează în volumul de vizibilitate  $[-1, 1]^3$ . Vertex-urile aflate în afara acestui volum, în spatele camerei sau în afara piramidei de vizibilitate, ar putea fi marcate cu un semnal dedicat și excluse de la maparea pe ecran, eliminând o categorie întreagă de rezultate eronate care în prezent nu sunt detectate de mecanismul de overflow.

**Saturarea coordonatelor de ecran** Chiar și cu clipping activ, erorile numerice acumulate pot produce coordonate finale ușor în afara rezoluției fizice a ecranului. Adăugarea unui stagiu de saturare după `ndc_to_screen_q`, care să limiteze  $x_s \in [0, W]$  și  $y_s \in [0, H]$ , ar preveni accesul în afara memoriei framebuffer și ar elimina artefactele vizuale cauzate de coordonate negative sau supradimensionate.

**Detecția underflow-ului** Implementarea curentă detectează depășirea superioară a domeniului numeric (overflow), însă nu și depășirea inferioară (underflow): situația în care rezultatul unui calcul este prea mic în valoare absolută pentru a fi reprezentat în formatul Q16.16 și este rotunjit la zero. Deși rezoluția de  $2^{-16}$  face underflow-ul mai puțin frecvent decât overflow-ul, acesta poate apărea în etapa de proiecție pentru vertex-uri foarte îndepărtate de cameră, unde  $f/z_r \approx 0$ , ducând la coordonate proiectate colapsate în originea ecranului.

**Generalizarea modelului de cameră** În implementarea curentă, camera este modelată exclusiv prin translație pe axa Z (`cam_z`). O îmbunătățire naturală ar fi extinderea la un model complet de cameră cu șase grade de libertate – translație pe toate cele trei axe și rotație proprie, implementat ca o etapă suplimentară de transformare în pipeline, înainte de rotația vertex-ului. Mergând mai departe, controlul dinamic al parametrilor camerei prin intrări externe (tastatură, UART) ar transforma sistemul dintr-un renderer static într-un vizualizator 3D interactiv, cu navigare în timp real în scena 3D.

## 5 CODUL SURSĂ AL PROIECTULUI

### 5.1 VPU\_Q.V

```

1 //-----
2 // Proiect      : Grafica 3D implementata pe FPGA
3 //
4 // Autor       : Petru-Andrei BRASOVEANU
5 // An         : 2026
6 //-----
7 // Descriere    : Vertex Processing Unit (VPU), sau Unitate de Procesare Vertex-uri
8 //               : Aceasta proceseaza calculele geometrice Rotatie - Proiectie - Mapare pe Ecran
9 //
10 //              : Reprezentare numerica: virgula fixa semnata (signed fixed-point)
11 //-----
12
13 module vpu_q #(
14     parameter INT_BITS = 16,                // Numar de biti parte intreaga (include semnul)
15     parameter FRAC_BITS = 16               // Numar de biti parte fractionara
16 ) (
17     input                clk,                // Semnal de ceas
18     input                rst_n,              // Reset asincron (activ in 0)
19     input                start,              // Pornire proces conversie
20
21     input                [2:0] rotation,     // Flag selectare tip de rotatie
22     input                [9:0] angle,        // Unghiul de rotatie
23
24     input                [INT_BITS+FRAC_BITS-1:0] f, x, y, z, w, h, cam_z, // Datele de intrare (NDC + dimensiuni
        ekran)
25
26     output reg [INT_BITS+FRAC_BITS-1:0] xs, ys, // Datele de iesire (coordonate ecran)
27     output reg valid, // Flag finalizare conversie
28     output reg overflow, // Indicator depasire domeniu numeric (DEBUG)
29     output reg [3:0] dbg_state // Flag stare FSM (DEBUG)
30 );
31
32 localparam WIDTH = INT_BITS + FRAC_BITS;
33
34 // -----
35 // Definitie stari FSM
36 // -----
37
38 localparam IDLE = 4'b0000, // Asteapta semnalul de start
39 LOAD = 4'b0001, // Incarca datele de intrare in registrele interne
40 START_ROT = 4'b0010,
41 CALC_ROT = 4'b0011, // Se calculeaza etapa de rotatie
42 DONE_ROT = 4'b0100, // Salvare rezultate rotatie
43 START_PROJ = 4'b0101,
44 CALC_PROJ = 4'b0110, // Se calculeaza etapa de proiectie
45 DONE_PROJ = 4'b0111, // Salvare rezultate proiectie
46 START_NDC = 4'b1000,
47 CALC_NDC = 4'b1001, // Se calculeaza etapa de mapare pe ecran
48 DONE = 4'b1010; // Rezultate finale valide
49
50 reg [3:0] state, next_state;
51 assign dbg_state = state;
52
53
54 // -----
55 // Registrele interne ptr pipeline
56 // -----
57
58 // Registre de intrare
59 reg [WIDTH-1:0] reg_x, reg_y, reg_z, reg_f, reg_w, reg_h, reg_cam_z;
60 reg [9:0] reg_angle;
61 reg [2:0] reg_rotation;
62
63 // Registre intermediare
64 reg [WIDTH-1:0] reg_xr, reg_yr, reg_zr; // Dupa rotatie
65 reg [WIDTH-1:0] reg_xp, reg_yp; // Dupa proiectie
66 reg ovf_rot_r, ovf_proj_r; // Acumulatori overflow intermediare
67
68
69 // -----
70 // Semnale interfata submodule aritmetice
71 // -----
72
73 reg rot_start, proj_start, ndc_start; // Pulsuri de start (1 ciclu)
74 wire rot_valid, proj_valid, ndc_valid; // Semnale rezultat valid
75
76 // Iesiri wire din submodule
77 wire [WIDTH-1:0] xr_w, yr_w, zr_w;
78 wire [WIDTH-1:0] xp_w, yp_w;

```

```

79 wire [WIDTH-1:0] xs_w, ys_w;
80
81 wire ovf_rot_w, ovf_proj_w, ovf_ndc_w; // Semnale overflow din submodule (DEBUG)
82
83
84
85
86 // -----
87 // Instantiere submodule aritmetice
88 // -----
89
90 // Submodul pentru calcul rotatie
91 rotation_q #(
92   .INT_BITS(INT_BITS),
93   .FRAC_BITS(FRAC_BITS)
94 ) u_rotation_q (
95   .clk(clk),
96   .rst_n(rst_n),
97   .start(rot_start),
98   .rotation(reg_rotation),
99   .x(reg_x), .y(reg_y), .z(reg_z),
100  .angle(reg_angle),
101  .xr(xr_w), .yr(yr_w), .zr(zr_w),
102  .valid(rot_valid),
103  .overflow(ovf_rot_w),
104  .dbg_state()
105 );
106
107 // Submodul pentru calcul proiectie
108 proj_q #(
109   .INT_BITS(INT_BITS),
110   .FRAC_BITS(FRAC_BITS)
111 ) u_proj_q (
112   .clk(clk),
113   .rst_n(rst_n),
114   .start(proj_start),
115   .f(reg_f), .x(reg_xr), .y(reg_yr), .z(reg_zr),
116   .xp(xp_w), .yp(yp_w),
117   .valid(proj_valid),
118   .overflow(ovf_proj_w),
119   .dbg_state()
120 );
121
122 // Submodul pentru calcul mapare pe ecran
123 ndc_to_screen_q #(
124   .INT_BITS(INT_BITS),
125   .FRAC_BITS(FRAC_BITS)
126 ) u_ndc_to_screen_q (
127   .clk(clk),
128   .rst_n(rst_n),
129   .start(ndc_start),
130   .xp(reg_xp), .yp(reg_yp), .w(reg_w), .h(reg_h),
131   .xs(xs_w), .ys(ys_w),
132   .valid(ndc_valid),
133   .overflow(ovf_ndc_w),
134   .dbg_state()
135 );
136
137
138 // -----
139 // Logica FSM - Calea de control
140 // -----
141
142 always @(posedge clk or negedge rst_n) begin
143   if (!rst_n) state <= IDLE;
144   else      state <= next_state;
145 end
146
147 always @(*) begin
148   case (state)
149     IDLE:      next_state = start ? LOAD : IDLE;
150     LOAD:      next_state = START_ROT;
151     START_ROT: next_state = CALC_ROT;
152     CALC_ROT:  next_state = rot_valid ? DONE_ROT : CALC_ROT;
153     DONE_ROT:  next_state = START_PROJ;
154     START_PROJ: next_state = CALC_PROJ;
155     CALC_PROJ: next_state = proj_valid ? DONE_PROJ : CALC_PROJ;
156     DONE_PROJ: next_state = START_NDC;
157     START_NDC: next_state = CALC_NDC;
158     CALC_NDC:  next_state = ndc_valid ? DONE : CALC_NDC;
159     DONE:      next_state = IDLE;
160     default:   next_state = IDLE;
161   endcase
162 end
163
164
165 // -----
166 // Logica FSM - Calea de date
167 // -----

```

```

168
169 always @(posedge clk or negedge rst_n) begin
170 if (!rst_n) begin
171 // Reset complet registre interne si iesiri
172
173 reg_x      <= 0; reg_y      <= 0; reg_z      <= 0;
174 reg_f      <= 0; reg_w      <= 0; reg_h      <= 0;
175 reg_angle   <= 0; reg_rotation <= 0;
176 reg_cam_z   <= 0;
177 reg_xr      <= 0; reg_yr    <= 0; reg_zr    <= 0;
178 reg_xp      <= 0; reg_yp    <= 0;
179
180 ovf_rot_r    <= 0; ovf_proj_r <= 0;
181
182 xs           <= 0;
183 ys           <= 0;
184
185 rot_start    <= 1'b0;
186 proj_start   <= 1'b0;
187 ndc_start    <= 1'b0;
188
189 valid        <= 1'b0;
190 overflow     <= 1'b0;
191 end else begin
192
193 case (state)
194
195 // Asteapta semnalul de start
196 IDLE: begin
197 valid        <= 1'b0; // Valid este 1 doar in DONE
198 rot_start    <= 1'b0;
199 proj_start   <= 1'b0;
200 ndc_start    <= 1'b0;
201 end
202
203 // Incarca datele de intrare in registre
204 LOAD: begin
205 reg_x        <= x;
206 reg_y        <= y;
207 reg_z        <= z;
208 reg_f        <= f;
209 reg_w        <= w;
210 reg_h        <= h;
211 reg_angle    <= angle;
212 reg_rotation <= rotation;
213 reg_cam_z    <= cam_z;
214 end
215
216 START_ROT: begin
217 rot_start    <= 1'b1; // Puls 1 ciclu catre rotation_q
218 end
219
220 CALC_ROT: begin
221 rot_start    <= 1'b0; // ptr a nu porni accidental submodulul din nou
222 end
223
224 DONE_ROT: begin
225 reg_xr      <= xr_w;
226 reg_yr      <= yr_w;
227 reg_zr      <= zr_w + reg_cam_z; // translatie camera pe Z
228 ovf_rot_r    <= ovf_rot_w;
229 end
230
231 START_PROJ: begin
232 proj_start   <= 1'b1; // Puls 1 ciclu catre proj_q
233 end
234
235 CALC_PROJ: begin
236 proj_start   <= 1'b0; // ptr a nu porni accidental submodulul din nou
237 end
238
239 DONE_PROJ: begin
240 reg_xp      <= xp_w;
241 reg_yp      <= yp_w;
242 ovf_proj_r   <= ovf_proj_w;
243 end
244
245 START_NDC: begin
246 ndc_start    <= 1'b1; // Puls 1 ciclu catre ndc_to_screen_q
247 end
248
249
250 CALC_NDC: begin
251 ndc_start    <= 1'b0; // ptr a nu porni accidental submodulul din nou
252 end
253
254 // Rezultat final disponibil
255 DONE: begin
256 xs          <= xs_w;

```

```

257 ys      <= ys_w;
258
259 overflow <= ovf_rot_r | ovf_proj_r | ovf_ndc_w;
260 valid <= 1'b1;
261 end
262 endcase
263 end
264 end
265
266 endmodule

```

## Secvență de Cod 1: Modulul DUT: Vertex Processing Unit

### 5.2 TB\_VPU\_Q.V

```

1  //-----
2  // Proiect      : Grafica 3D implementata pe FPGA
3  //
4  // Autor       : Petru-Andrei BRASOVEANU
5  // An         : 2026
6  //-----
7  // Descriere    : Testbench pentru Vertex Processing Unit (VPU)
8  //              : Verifica functionalitatea etapelor de Rotatie, Proiectie
9  //              : si Mapare prin comparatie cu un model de referinta (Golden Model)
10 //-----
11
12 `timescale 1ns/1ps
13
14 module tb_vpu_q;
15
16 // -----
17 // Parametri de configurare (NU ATINGE)
18 // -----
19
20 parameter PER          = 8;           // Perioada ceasului (8ns => 125MHz)
21 parameter INT_BITS    = 16;          // Biti pentru partea intreaga (semnati)
22 parameter FRAC_BITS   = 16;          // Biti pentru partea fractionara
23 parameter WIDTH       = INT_BITS + FRAC_BITS; // Latime totala cuvnt
24 parameter MAX_R       = 32'h7111_1111; // Valoare maxima pentru saturatie
25 parameter MIN_R       = 32'h8000_0000; // Valoare minima pentru saturatie
26 parameter real SCALE  = 2.0**FRAC_BITS; // Factor de scalare pentru Q16.16 (2^FRAC_BITS)
27
28 // -----
29 // Parametri modificabili
30 // -----
31
32 parameter NUM_RAND    = 2000;        // Numarul de teste aleatorii
33 parameter THRESHOLD   = 2;           // Toleranta acceptata pentru erori (pixeli)
34
35 // Constante geometrice in format Q16.16
36 parameter CAMERA_Z    = 32'h0001_8000; // Distanța camera = 1.5
37 parameter FOCAL LENGHT = 32'h0000_0666; // Lungime focala = 0.05 (aprox 50mm)
38 parameter SCREEN_WIDTH = 32'h0140_0000; // Latime ecran = 320.0
39 parameter SCREEN_HEIGHT = 32'h00f0_0000; // Inaltime ecran = 240.0
40
41 // Setari pentru precizia datelor generate aleatoriu
42 integer precision_x_int = 12;
43 integer precision_x_frac = 16;
44
45 integer precision_y_int = 12;
46 integer precision_y_frac = 16;
47
48 integer precision_z_int = 1;
49 integer precision_z_frac = 16;
50
51 // -----
52 // Semnale de interfata
53 // -----
54
55 wire clk, rst_n;
56 reg start; // Semnal start catre DUT
57 wire valid; // Semnal valid de la DUT
58
59 // Registre pentru intrarile DUT
60 reg [2:0] rotation; // Tipul de rotatie (X, Y, Z)
61 reg [9:0] angle; // Unghiul de rotatie
62 reg [WIDTH-1:0] f, x, y, z, w, h; // Parametrii geometrici
63 reg [WIDTH-1:0] cam_z; // Pozitia camerei
64
65 // Fire pentru iesirile DUT
66 wire [WIDTH-1:0] xs, ys; // Coordonate ecran rezultate

```

```

70 wire [3:0]      dbg_state;           // Starea interna a FSM-ului DUT
71 wire           overflow;             // Indicator depasire domeniu numeric
72
73 // Variabile statistice pentru raportare
74 integer error_count = 0;             // Numar de erori detectate
75 integer overflow_count = 0;          // Numar de cazuri cu overflow
76 integer test_count = 0;              // Numar total de teste rulate
77 real error_rate;                     // Rata de eroare procentuala
78
79
80
81 // -----
82 // Ceas
83 // -----
84
85 ck_rst_tb #(
86   .CK_SEMIPERIOD(PER/2)
87 ) clk_gen (
88   .clk(clk),
89   .rst_n(rst_n)
90 );
91
92
93 // -----
94 // DUT
95 // -----
96
97 vpu_q #(
98   .INT_BITS(INT_BITS),
99   .FRAC_BITS(FRAC_BITS)
100 ) dut (
101   .clk(clk),
102   .rst_n(rst_n),
103   .start(start),
104   .rotation(rotation),
105   .angle(angle),
106   .f(f), .x(x), .y(y), .z(z), .w(w), .h(h),
107   .cam_z(cam_z),
108   .xs(xs), .ys(ys),
109   .valid(valid),
110   .overflow(overflow),
111   .dbg_state(dbg_state)
112 );
113
114
115 // -----
116 // Task 1: Start operatie
117 // -----
118
119 // Lanseaza un ciclu de procesare prin activarea semnalului start
120 task start_op(
121   input [2:0] in_rot, input [9:0] in_ang,
122   input [WIDTH-1:0] in_w, in_h, in_f, in_x, in_y, in_z, in_cam_z
123 );
124 begin
125   // initializam datele de intrare
126   angle = in_ang;
127   rotation = in_rot;
128   w = in_w;
129   h = in_h;
130   f = in_f;
131   x = in_x;
132   y = in_y;
133   z = in_z;
134   cam_z = in_cam_z;
135
136   start = 1'b1; // Activeaza start
137   @(posedge clk); #1;
138   start = 1'b0; // Dezactiveaza start dupa un ciclu
139 end
140 endtask
141
142
143 // -----
144 // Task 2: Definirea Modelului de Referinta (Task-ul de calcul)
145 // -----
146
147 // Calculeaza rezultatul ideal folosind aritmetica 'real' (floating point)
148 task calculate_vpu_gold(
149   input [2:0] r_rot, input [9:0] r_ang,
150   input real rx, input real ry, input real rz,
151   input real rf, input real rw, input real rh,
152   input real r_cam_z,
153   output real exp_xs_r, output real exp_ys_r
154 );
155   real xr, yr, zr, xp, yp;
156   real ra, r_sin, r_cos;
157 begin

```

```

159 // ETAPA 1: ROTATIE
160 ra = (r_ang / 2.0) * (3.14159265 / 180.0); // exprimata in radiani
161 r_sin = $sin(ra); r_cos = $cos(ra);
162
163
164 case (r_rot)
165 3'b000: begin xr = rx; yr = ry * r_cos - rz * r_sin; zr = ry * r_sin + rz * r_cos; end
166 3'b001: begin xr = rx; yr = ry * r_cos + rz * r_sin; zr = -ry * r_sin + rz * r_cos; end
167 3'b010: begin xr = rx * r_cos + rz * r_sin; yr = ry; zr = -rx * r_sin + rz * r_cos; end
168 3'b011: begin xr = rx * r_cos - rz * r_sin; yr = ry; zr = rx * r_sin + rz * r_cos; end
169 3'b100: begin xr = rx * r_cos - ry * r_sin; yr = rx * r_sin + ry * r_cos; zr = rz; end
170 3'b101: begin xr = rx * r_cos + ry * r_sin; yr = -rx * r_sin + ry * r_cos; zr = rz; end
171 default: begin xr = rx; yr = ry; zr = rz; end
172 endcase
173
174 zr = zr + r_cam_z; // Translatie pe axa Z (pozitia camerei)
175
176 // ETAPA 2: PROIECTIE
177 if (zr == 0.0) begin
178 xp = (rf >= 0.0) ? MAX_R : MIN_R;
179 yp = (rf >= 0.0) ? MAX_R : MIN_R;
180 end else begin
181 xp = xr * (rf / zr); // Formula proiectie X
182 yp = yr * (rf / zr); // Formula proiectie Y
183 end
184
185 // --- ETAPA 3: NDC TO SCREEN ---
186 exp_xs_r = (xp * rh + rw) / 2.0; // Mapare X pe latime
187 exp_ys_r = (rh - yp * rh) / 2.0; // Mapare Y cu inversare (axa Y ecran e in jos)
188 end
189 endtask
190
191
192 // -----
193 // Task: Generator numere in virgula fixa
194 // -----
195
196 // Produce o valoare aleatorie incadrata in bitii de precizie specificati
197 task rand_fp;
198 input integer rand_int_bits;
199 input integer rand_frac_bits;
200 output reg [WIDTH-1:0] result;
201
202 reg signed [31:0] raw;
203 integer rand_max;
204 begin
205
206 rand_max = 1 << (rand_int_bits + rand_frac_bits - 1);
207 raw = $random % rand_max; // Generare valoare bruta cu semn
208
209 // Aliniere la formatul Q16.16: se shifteaza pentru a umple partea fractionara
210 result = $signed(raw) << (FRAC_BITS - rand_frac_bits);
211 end
212 endtask
213
214
215 // -----
216 // Task 3: Un singur test random (intern)
217 // -----
218
219 // Ruleaza o singura iteratie: calculeaza gold, ruleaza DUT, compara
220 task run_one_random;
221 input [2:0] r_rot;
222 input [9:0] r_ang;
223 input [WIDTH-1:0] r_w, r_h, r_f, r_x, r_y, r_z;
224 input [WIDTH-1:0] r_cam_z;
225 input integer idx;
226
227 real rw_r, rh_r, rf_r, rx_r, ry_r, rz_r, ex_xs, ex_ys, r_cam_z_r;
228 reg signed [WIDTH-1:0] v_exp_xs, v_exp_ys;
229
230 integer diff_x, diff_y;
231 real err_x, err_y;
232 begin
233
234 // 1. Conversie din Fixed-Point (DUT) in Real (Golden Model)
235 r_cam_z_r = $itor($signed(r_cam_z)) / SCALE;
236 rf_r = $itor($signed(r_f)) / SCALE;
237 rx_r = $itor($signed(r_x)) / SCALE;
238 ry_r = $itor($signed(r_y)) / SCALE;
239 rz_r = $itor($signed(r_z)) / SCALE;
240 rw_r = $itor($signed(r_w)) / SCALE;
241 rh_r = $itor($signed(r_h)) / SCALE;
242
243
244 // 2. Calcul model de referinta
245 calculate_vpu_gold(r_rot, r_ang, rx_r, ry_r, rz_r, rf_r, rw_r, rh_r, r_cam_z_r, ex_xs, ex_ys);
246
247 // 3. Conversie inapoi in Fixed-Point pentru comparatie directa

```



```

248 v_exp_xs = $rtoi(ex_xs * SCALE);
249 v_exp_ys = $rtoi(ex_ys * SCALE);
250
251
252 // 4. Stimulare DUT si asteptare semnal valid
253 start_op(r_rot, r_ang, r_w, r_h, r_f, r_x, r_y, r_z, r_cam_z);
254 wait(valid == 1'b1);
255 @(posedge clk); #2;
256
257
258 // 5. Verificare eroare fata de pragul THRESHOLD
259 diff_x = $signed(xs) - $signed(v_exp_xs);
260 diff_y = $signed(ys) - $signed(v_exp_ys);
261
262 err_x = $itor(diff_x)/SCALE; // Eroare X in unitati reale (pixeli)
263 err_y = $itor(diff_y)/SCALE; // Eroare Y in unitati reale (pixeli)
264
265 $display("-----");
266 $display("TEST NR. %0d: Rot = %0d Ang = %0.1f | f = %h x = %h y = %h z = %h",
267 idx, r_rot, r_ang, r_f, r_x, r_y, r_z);
268
269 if (overflow) begin
270 $display("SKIP [%0d]: Overflow detectat", idx);
271 overflow_count = overflow_count + 1;
272 end else if ((err_x > THRESHOLD) || (err_x < -THRESHOLD) || (err_y > THRESHOLD) || (err_y < -
    THRESHOLD)) begin
273 error_count = error_count + 1;
274 $display("EROARE: xs=%h(exp %h, DIFERENTA=%f pixeli)", xs, v_exp_xs, (err_x < 0 ? -err_x : err_x));
275 $display("ys=%h(exp %h, DIFERENTA=%f pixeli)", ys, v_exp_ys, (err_y < 0 ? -err_y : err_y));
276 end else begin
277 $display("OK [%0d]: ScreenX=%h ScreenY=%h", idx, xs, ys);
278 end
279 end
280 endtask
281
282
283 // -----
284 // Task: Bucla de teste aleatorii
285 // -----
286 task run_random_tests;
287 input integer seed;
288 integer i;
289 reg [WIDTH-1:0] rf_t, rx_t, ry_t, rz_t, rw_t, rh_t;
290 reg [9:0] ang_t;
291 reg [2:0] rot_t;
292 begin
293 $display("--- START TESTE RANDOM ---");
294
295 for (i = 0; i < NUM_RAND; i = i + 1) begin
296 // Initializare parametri fixi ptr ecran
297 rw_t = SCREEN_WIDTH;
298 rh_t = SCREEN_HEIGHT;
299 rf_t = FOCAL LENGHT;
300
301 // Generare unghi si tip rotatie
302 rot_t = $urandom % 6;
303 ang_t = $urandom % 720;
304
305 // Generare coordonate Vertex aleatorii
306 rand_fp(precision_x_int, precision_x_frac, rx_t);
307 rand_fp(precision_y_int, precision_y_frac, ry_t);
308 rand_fp(precision_z_int, precision_z_frac, rz_t);
309
310 run_one_random(rot_t, ang_t, rw_t, rh_t, rf_t, rx_t, ry_t, rz_t, CAMERA_Z, i);
311 test_count = test_count + 1;
312 end
313
314 $display("--- SFARSIT TESTE RANDOM (seed=%0d) ---", seed);
315 end
316 endtask
317
318
319 // -----
320 // Subrutina principala
321 // -----
322
323 integer file_out; // Descriptorul de fisier
324
325 initial begin
326
327 // Deschide fisierul pentru scriere
328 file_out = $fopen("rezultate_test_vpu.txt", "w");
329
330 // aVerific a dac sfiierul a fost deschis corect
331 if (file_out == 0) begin
332 $display("EROARE: Nu s-a putut crea fisierul de iesire!");
333 $finish;
334 end
335 end

```

```

336 $display("=== START TEST VERTEX PROCESSING UNIT ===");
337
338
339 // Initializare semnale si registre
340 start = 0; cam_z = CAMERA_Z;
341 h = 0; w = 0; f = 0; angle = 0; rotation = 0; x = 0; y = 0; z = 0;
342
343 repeat(5) @(posedge clk);
344
345 // Rularea testelor
346 run_random_tests(1); // seed fix => reproductibil
347
348 // Calcul statistici finale
349 error_rate = (error_count * 100.0) / (test_count - overflow_count);
350
351 // Scriem in fisier folosind $fdisplay
352 $fdisplay(file_out, "-----");
353 $fdisplay(file_out, "=== TEST VPU FINALIZAT ===");
354 $fdisplay(file_out, "Frecventa ceas = %0d MHz", (1/ $itor(PER)) * 1000);
355 $fdisplay(file_out, "Format Q      = Q%0d.%0d semnat", INT_BITS, FRAC_BITS);
356 $fdisplay(file_out, "Focala      = %0g mm", ($itor(FOCAL_LENGTH) / SCALE) * 1000);
357 $fdisplay(file_out, "Latime ecran = %0d px", $itor(SCREEN_WIDTH) / SCALE);
358 $fdisplay(file_out, "Inaltime ecran = %0d px", $itor(SCREEN_HEIGHT) / SCALE);
359 $fdisplay(file_out, "Biti random X = %0d INT, %0d FRAC", precision_x_int, precision_x_frac);
360 $fdisplay(file_out, "Biti random Y = %0d INT, %0d FRAC", precision_y_int, precision_y_frac);
361 $fdisplay(file_out, "Biti random Z = %0d INT, %0d FRAC", precision_z_int, precision_z_frac);
362 $fdisplay(file_out, "");
363 $fdisplay(file_out, "Prag eroare   = %g pixeli", THRESHOLD);
364 $fdisplay(file_out, "Teste totale  = %0d", test_count);
365 $fdisplay(file_out, "Erori        = %0d", error_count);
366 $fdisplay(file_out, "Overflow-uri  = %0d", overflow_count);
367 $fdisplay(file_out, "RATA DE ERORI = %f %%", error_rate);
368 $fdisplay(file_out, "-----");
369
370 // Recomandat: lasa si display-urile normale ca sa vezi rezultatul si in consola simulatorului
371 $display("=== REZULTATE EXPORTATE IN rezultate_test_vpu.txt ===");
372
373 // Inchide fisierul
374 $fclose(file_out);
375 $finish;
376 end
377
378 endmodule
379

```

## Secvență de Cod 2: Testbench pentru simulare

# Bibliografie

- [1] C. Inacio and D. Ombres, "The dsp decision: Fixed point or floating?" *IEEE Spectrum*, vol. 33, no. 9, pp. 72–74, 1996. DOI: 10.1109/6.535397.
- [2] Khronos Group, *Rendering pipeline overview*, [https://wikis.khronos.org/opengl/Rendering\\_Pipeline\\_Overview](https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview), Accessed: 2026-04-25, 2022.
- [3] National Instruments, *Advantages of xilinx 7 series fpga and soc devices*, <https://www.ni.com/en/shop/compactrio/what-are-compactrio-controllers/advantages-of-xilinx-7-series-fpga-and-soc-devices.html>, Accessed: 2026-04-25, 2024.
- [4] MiSTer-devel Community, *Mister fpga wiki*, [https://github.com/MiSTer-devel/Wiki\\_MiSTer/wiki](https://github.com/MiSTer-devel/Wiki_MiSTer/wiki), Accessed: 2026-04-25, 2025.
- [5] SpinalHDL Project, *Spinalhdl documentation*, <https://spinalhdl.github.io/SpinalDoc-RTD/master/index.html>, Accessed: 2026-04-25, 2025.
- [6] K. Kim, Hoosung-Lee, S. Cho, and S. Park, "Implementation of 3d graphics accelerator using full pipeline scheme on fpga," in *2008 International SoC Design Conference*, vol. 02, 2008, pp. 11-97-11-100. DOI: 10.1109/SOCD.2008.4815693.
- [7] F. Bensaali, A. Amira, I. Uzun, and A. Ahmedsaid, "An fpga implementation of 3d affine transformations," in *10th IEEE International Conference on Electronics, Circuits and Systems, 2003. ICECS 2003. Proceedings of the 2003*, vol. 2, 2003, 715–718 Vol.2. DOI: 10.1109/ICECS.2003.1301885.
- [8] R. Saha, N. U. Sadad, and M. N. I. Mondal, "Designing a graphics processing unit for 2d rendering on fpga for educational purpose," in *2024 3rd International Conference on Advancement in Electrical and Electronic Engineering (ICAEEE)*, 2024, pp. 1–6. DOI: 10.1109/ICAEEE62219.2024.10561635.